

VR Chemistry Lab – System Architecture

Reference Client & Server Implementation

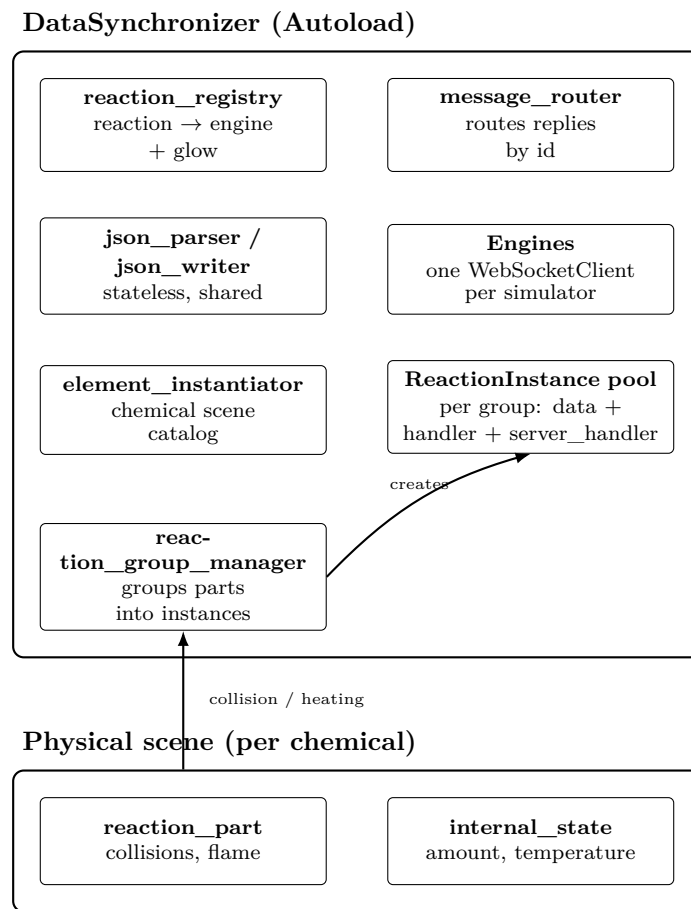
This document describes the current code, both in Godot and Python. It describes one concrete implementation, not a contract.

For Websocket contracts there is also the *VR Chemistry Lab – WebSocket API Reference* and for users, both looking to install and use the Lab, as well as expand the frontend, there is the *emphVR Chemistry Lab – Handbook*.

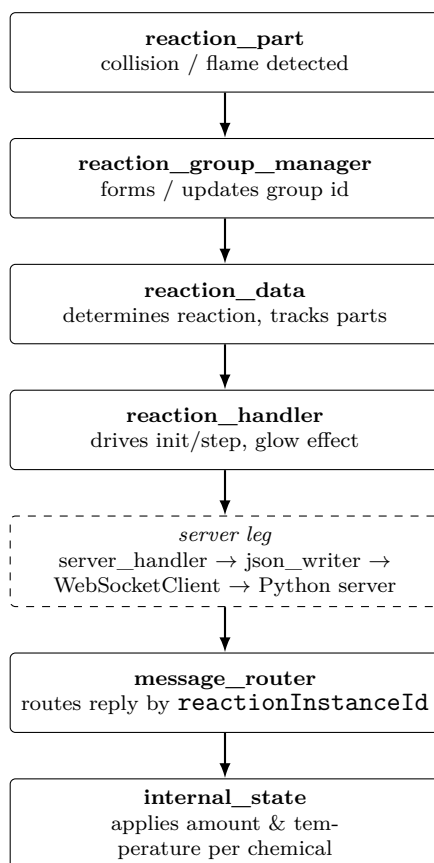
Contents

1	Architecture Overview	2
2	Godot Client – Reaction Scene Scripts	4
2.1	DataSynchronizer (Autoload/Singleton) – <code>data_synchronizer.gd</code>	4
2.1.1	Child node: ElementInstantiator – <code>element_instantiator.gd</code>	4
2.2	ReactionRegistry – <code>reaction_registry.gd</code>	5
2.3	ReactionGroupManager – <code>reaction_group_manager.gd</code>	5
2.4	ReactionData – <code>reaction_data.gd</code>	6
2.5	ReactionHandler – <code>reaction_handler.gd</code>	6
2.6	ReactionPart – <code>reaction_part.gd</code>	7
2.7	InternalState – <code>internal_state.gd</code>	7
3	Godot Client – Transport Scripts	9
3.1	ServerHandler – <code>server_handler.gd</code>	9
3.2	MessageRouter – <code>message_router.gd</code>	10
3.3	JsonWriter / JsonParser – <code>json_writer.gd</code> , <code>json_parser.gd</code>	10
3.4	WebSocketClient – <code>websocketClient.gd</code>	10
4	Python Server – Module Overview	12

1 Architecture Overview



Message round-trip within one group – from physical contact to the resulting state update, including the wire-protocol leg:



Everything inside the dashed *server leg* box is the wire protocol: its exact message shapes, methods, and rules are specified in the API Reference document, not here.

Call chain (short form):

```

ReactionPart → ReactionGroupManager → DataSynchronizer → ReactionInstance (ReactionData
+ ReactionHandler) ↔ InternalState
ReactionHandler ↔ ServerHandler ↔ JsonWriter/JsonParser ↔ WebSocketClient ↔
MessageRouter
  
```

2 Godot Client – Reaction Scene Scripts

These scripts model the physical reaction scene: which chemicals are present, how they are grouped into reactions, and how their visual state (glow, mesh scale) reacts to simulation data. None of them talk to the network directly.

2.1 DataSynchronizer (Autoload/Singleton) – `data_synchronizer.gd`

Central hub of the system: owns the reaction registry, the pool of active `ReactionInstances`, the message router, and the engine connections. Almost every other script reaches other parts of the system through this autoload.

Signal	Description
<code>dataChanged(data: Dictionary)</code>	Emitted once per process. <code>data</code> , the current reaction step for all reactions, which contains <code>reactionIds</code> and <code>amount_of_*</code> entries. Subscribed to by <code>InternalState</code> and <code>ReactionHandler</code> .

Method	Description
<code>getReactionStrength(reaction: String) -> float</code>	Reads a reaction's glow intensity from the registry (0.0 = no glow). Used by <code>ReactionHandler</code> .
<code>createReactionInstance(group Id: String) -> Node</code>	Instantiates a new <code>ReactionInstance</code> (holding <code>ReactionData</code> + <code>ReactionHandler</code> + <code>server_handler</code>) for a group. Called by <code>ReactionGroupManager</code> .
<code>removeReactionInstance(group Id: String) -> void</code>	Removes a <code>ReactionInstance</code> from the pool again.
<code>instantiate(world: Node, element: String) -> Node</code>	Delegates to <code>ElementInstantiator</code> , spawns the 3D scene of a chemical.
<code>setStepOfCompleteReaction (dataStep: Dictionary) -> void</code>	Fires <code>dataChanged</code> with one simulation step. Called by <code>ReactionHandler</code> once new data is available.
<code>getEngineConnectionFor Reaction(reaction: String) -> Node</code>	Resolves the WebSocket engine node responsible for a reaction, via <code>reactionRegistry</code> . Used by <code>server_handler</code> .

Field	Description
<code>reactionRegistry: Reaction Registry</code>	Central configuration, loaded at runtime (see below).
<code>reactionGroupManager: Node</code>	Reference to the <code>reactionGroupManager</code> child node.
<code>EXISTENCE_TRESHHOLD_IN_MOL: float</code>	Threshold below which an amount of substance counts as "depleted".

2.1.1 Child node: *ElementInstantiator* – `element_instantiator.gd`

Small catalog + factory for chemical scenes, owned by `DataSynchronizer` and only ever called through it (`DataSynchronizer.instantiate`) – it has no independent role in the system.

Method	Description
<code>addAllChemicalsToDictionary()</code> -> void	Scans <code>res://objects/chemicals</code> on startup and indexes all chemical scenes by name.
<code>instantiate(parent: Node, element: String)</code> -> Node	Instantiates a chemical's scene under parent.

2.2 ReactionRegistry – `reaction_registry.gd`

Runtime-loaded, single source of truth for "which reactions exist, which engine simulates them, how strongly they glow". Generated from the Python/YAML reaction source – never hand-edited in Godot.

Method	Description
<code>loadFromFile(path: String)</code> -> bool	Loads the JSON configuration (engine assignment, glow strength, aqueous reagents). Keeps the previous entries on failure.
<code>findEntry(reaction: String)</code> -> Dictionary	Returns the matching registry entry (exact or prefix match). Empty dictionary if nothing matches.
<code>findReactionByReactants</code> (parts: Dictionary) -> String	Derives the matching <code>whatReaction</code> key from the currently present reaction parts. Used by <code>ReactionData</code> .

Field	Description
<code>entries: Array</code>	List of all registry entries.
<code>aqueousReagents: Array</code>	Names of all aqueous reagents.

2.3 ReactionGroupManager – `reaction_group_manager.gd`

Dynamically resolves which `ReactionInstance` a `ReactionPart` currently belongs to – forming, merging and tearing down groups as parts collide, appear, separate or deplete.

Method	Description
<code>onPartsCollided(partA, partB: ReactionPart)</code> -> void	Called when two parts physically collide. Creates a new group if needed, or merges two existing groups. Caller: <code>ReactionPart</code> .
<code>onPartAppeared(part: ReactionPart)</code> -> void	A part appears without a collision partner (e.g. heating). Creates a new group if needed.
<code>onPartLeftReaction(part: ReactionPart)</code> -> void	Part physically separates but keeps existing. Gets queued for removal.
<code>onPartDepleted(part: ReactionPart)</code> -> void	Part is consumed and gets deleted.
<code>getInstanceForPart(part: ReactionPart)</code> -> Node	Returns the <code>ReactionInstance</code> a part currently belongs to.
<code>getReactionHandlerForPart</code> (part: ReactionPart) -> Node	Alias for <code>getInstanceForPart</code> . Used by <code>ReactionPart</code> when starting/stopping heating.

Method	Description
<code>getActiveGroupCount() -> int</code>	Number of active groups (debug).
<code>getGroupIdForPart(part: ReactionPart) -> String</code>	Group id of a part (debug).

2.4 ReactionData – `reaction_data.gd`

Child node of a `ReactionInstance`. Holds the chemicals currently involved in one group and derives the resulting reaction key (`whatReaction`) from their composition.

Method	Description
<code>addReactionPart(part: ReactionPart) -> void</code>	Adds a part, updates <code>whatReaction</code> and position.
<code>queueReactionPartToRemove(part: ReactionPart) -> void</code>	Queues a part for removal (processed on the next <code>_process</code>).
<code>lastRemovedCheckedNowSetToNull() -> void</code>	Clears the list of most recently removed chemicals.
<code>getWhatReaction() -> String</code>	Current reaction key, or "".
<code>getReactionPart(identifier: String) -> ReactionPart</code>	Part by chemical name.
<code>isChemicalPartOfReaction(identifier: String) -> bool</code>	–
<code>isChemicalPartOfLastRemoved(identifier: String) -> bool</code>	–
<code>reactionDetected() -> bool</code>	true once <code>whatReaction</code> is set.
<code>getReactingPosition() -> Vector3</code>	Centroid of all involved parts.
<code>getReactionIDs() -> Dictionary</code>	Chemical name → <code>instance_id</code> .
<code>getCompositionVersion() -> int</code>	Counts every change to the composition (drives the glow trigger).
<code>getReactionTemperature() -> float</code>	Highest temperature among all parts.
<code>getReactionAmountInMol() -> Dictionary</code>	Amount of substance per chemical.

2.5 ReactionHandler – `reaction_handler.gd`

Child node of a `ReactionInstance`. Drives the simulation lifecycle (init/heat/step) and the glow effect via `server_handler`, and instantiates new reaction parts as they show up in simulation data.

Method	Description
<code>addReactionPart(part: ReactionPart) -> void</code>	Delegates to <code>reactionData</code> , resets the initialization state when the reaction changes.
<code>queueReactionPartToRemove(part: ReactionPart) -> void</code>	Delegates to <code>reactionData</code> .

Method	Description
<code>setJustStartedHeating(value: bool) -> void</code>	Setter; triggers the start-heating request on the next <code>_process</code> .
<code>setJustStoppedHeating(value: bool) -> void</code>	Same, for stopping heating.
<code>setSimulationData(data: Array) -> void</code>	Accepts <code>[dataBlockList, reactionIds]</code> from <code>server_handler</code> , discards stale data.
<code>initializeReaction(...), startHeatingAndRequestReactionStep(...), stopHeatingAndRequestReactionStep(...), requestReactionStep(...)</code>	Thin wrappers that call the matching <code>server_handler.request*</code> method (see Section 3).
<code>_on_reactionPartDepleted (part: ReactionPart) -> void</code>	Callback from <code>ReactionGroupManager</code> : queues the part for removal and deletes its scene.

2.6 ReactionPart – `reaction_part.gd`

Attached to every chemical instance in the 3D scene. Detects physical collisions and flame contact, and forwards the resulting state changes to `ReactionGroupManager` and its own `InternalState`.

Method	Description
<code>getState() -> InternalState</code>	Access to the associated <code>InternalState</code> .
<code>getChemicalName() -> String</code>	gets the chemical name identifier
<code>addSelfToReaction() -> void</code>	Registers the part with <code>ReactionGroupManager</code> initially (<code>onPartAppeared</code>).
<code>setIsInFlame(value: bool) -> void</code>	Starts/stops heating, reports group and heating status.
<code>getIsInFlame() -> bool</code>	if the <code>reactionPart</code> is currently in a flame (that would heat it)

Field	Description
<code>isPartOfReaction: bool</code>	Setter connects/disconnects <code>InternalState</code> to <code>DataSynchronizer.dataChanged</code> .

2.7 InternalState – `internal_state.gd`

Holds the physical state of a single chemical (temperature, amount, mesh scaling) and reacts to global simulation steps broadcast via `DataSynchronizer.dataChanged`.

Signal	Description
<code>reactionPartDepleted(reactionPart: ReactionPart)</code>	Fires once the amount of substance falls below the existence threshold. Subscribed to by <code>ReactionGroupManager</code> .

Method	Description
<code>connectToDataSynchronizer()</code> <code>-> void</code>	Subscribes to <code>DataSynchronizer.dataChanged</code> .
<code>disconnectFromDataSynchronizer()</code> <code>-> void</code>	Unsubscribes and notifies <code>ReactionGroupManager</code> (on <code>PartLeftReaction</code>).
<code>getTemperature()</code> <code>-> float</code> <code>/ addToTemperature(delta: float)</code>	—
<code>getCurrentAmountInMol()</code> <code>-> float</code> <code>/ setCurrentAmountInMol(amount: float)</code>	Scales the mesh size for precipitates and checks the existence threshold.
<code>equals(otherState: InternalState)</code> <code>-> bool</code>	Compares temperature and amount.
<code>isAffectedByData(data: Dictionary)</code> <code>-> bool</code>	Checks whether a simulation step concerns this chemical.
<code>setEffectsFromData(data: Dictionary)</code> <code>-> void</code>	Applies temperature/amount from a simulation step.

3 Godot Client – Transport Scripts

These scripts implement the client side of the wire protocol specified in the API Reference document: turning `ReactionHandler` requests into JSON, sending/receiving over WebSocket, and routing replies back to the right session. The protocol-level rules they enforce (one in-flight request per session, connection-ready precondition, `closeSession` bypassing the busy state) are specified as part of the contract in the API Reference, *Concurrency & Request Constraints*; this section only describes how the reference client happens to implement them.

3.1 ServerHandler – `server_handler.gd`

One instance per `ReactionInstance` (== per group). Owns that group’s session state (`reactionInstanceId`, whether/what is initialized, simulated time) and is the only script that decides whether a request may be sent right now.

Method	Description
<code>onMessageFromServer(text: String) -> void</code>	Entry point for incoming replies (called by <code>MessageRouter</code>). Parses text via <code>json_parser</code> , filters the result down to entries that carry a <code>timestamp</code> (acks like <code>initSuccess</code> are dropped here), and forwards playback steps to <code>ReactionHandler.setSimulationData</code> .
<code>sendMessageToServer(message: String, engine: Node) -> void</code>	Marks the session busy and calls <code>engine.sendMessage(message)</code> .
<code>requestReactionInit(reaction, reactionIds, temperature, amountInMol) -> bool</code>	Sends <code>methodName: "init"</code> . Fails silently (returns <code>false</code>) while busy, if <code>reactionIds</code> is empty, or if the engine isn’t ready yet.
<code>requestStartHeatingAndReactionStep(reaction, reactionIds) -> bool</code>	Sends <code>startHeatingAndRunUntilTargetTime</code> .
<code>requestStopHeatingAndReactionStep(reaction, reactionIds) -> bool</code>	Sends <code>stopHeatingAndRunUntilTargetTime</code> .
<code>requestReactionStep(reaction, reactionIds) -> bool</code>	Sends <code>runUntilTargetTime</code> .
<code>requestCloseSession() -> void</code>	Sends <code>closeSession</code> for the currently initialized reaction, bypassing the busy flag (teardown must not be blocked).
<code>isReactionInitialized(reaction, reactionIds) -> bool</code>	Compares against the locally cached initialized reaction + ids.
<code>hasInitializedReaction() -> bool / getInitializedReaction() -> String</code>	–
<code>resetReactionInitialized() -> void</code>	Closes the current session server-side first (important when the new formula runs on a <i>different</i> engine than the old one), then clears local state.
<code>getCurrentAbsolutReactionTimeInMs() -> float</code>	Locally tracked simulated time, advanced every <code>_process</code> .

3.2 MessageRouter – message_router.gd

Stateless dispatcher: the single place that maps an incoming server message back to the `server_handler` of the right group, via `reactionInstanceId`.

Method	Description
<code>registerServerHandler(serverHandler: Node, reactionInstanceId: String) -> void</code>	Called by <code>DataSynchronizer.createReactionInstance</code> .
<code>unregisterServerHandler(reactionInstanceId: String) -> void</code>	Called by <code>DataSynchronizer.removeReactionInstance</code> .
<code>onMessageFromServer(text: String) -> void</code>	Connected to every engine's <code>message_received</code> signal. Extracts <code>reactionInstanceId</code> and forwards <code>text</code> to the matching handler; silently drops messages with no registered handler.

3.3 JsonWriter / JsonParser – json_writer.gd, json_parser.gd

Stateless, shared across all engines/sessions. Own the exact request/ response shapes specified in the API Reference document.

Method	Description
<code>JsonWriter.getJsonFromDataForInit(whatReaction, reactionIds, targetTime, temperature, reactionInstanceId, amountInMol) -> String</code>	Builds the init request.
<code>JsonWriter.getJsonFromDataForReaction(whatReaction, reactionIds, targetTime, methodName, reactionInstanceId) -> String</code>	Builds any non-init request (<code>runUntilTargetTime</code> , <code>startHeating...</code> , <code>stopHeating...</code> , <code>closeSession</code>).
<code>JsonParser.getData(jsonData) -> [dataArray, reactionIds]</code>	Parses the response envelope, logs <code>success: false</code> errors, and returns <code>data.dataBlockList</code> together with <code>reactionIds</code> regardless of success. Returns null on unparsable input. It keeps every entry of <code>dataBlockList</code> – filtering out non-playback acks (see above) is left to <code>server_handler</code> .

3.4 WebSocketClient – websocketClient.gd

One instance per engine (child of an `Engines/<engineId>` node). Thin wrapper around Godot's WebSocket API that guarantees complete JSON objects are emitted even if the transport merges or splits messages.

Signal	Description
<code>message_received(text: String)</code>	Emitted once per complete JSON object received. <code>MessageRouter</code> connects to this for every engine in <code>DataSynchronizer._ready()</code> .

Method	Description
<code>sendMessage(message: String) -> void</code>	Sends raw text over the socket.
<code>getIsReady() -> bool</code>	Whether the connection handshake has completed. <code>server_handler</code> checks this before sending <code>init</code> .

4 Python Server – Module Overview

Server-side counterpart to the transport scripts above. Not part of the network contract either – included here purely so the error origins and extension points referenced from the API Reference document can be traced back to a concrete file.

Module	Responsibility
<code>websocket_server.py</code>	Accepts WebSocket connections, forwards raw text to <code>simulation_hub.py</code> , catches unexpected exceptions (<code>INTERNAL_ERROR</code>) and <code>None</code> handler results (<code>NULL_OUTPUT</code>).
<code>simulation_hub.py</code>	Parses the request envelope, resolves <code>whatReaction</code> to a simulation class (<code>getSimulationClassFromString</code>), dispatches <code>methodName</code> to the matching handler, and builds the response envelope (<code>buildResponseEnvelope()</code>). Raises <code>MISSING_REACTION_INSTANCE_ID</code> , <code>SIM_NOT_INITIALIZED</code> , <code>UNKNOWN_METHOD</code> , and <code>UNKNOWN_REACTION</code> .
<code>simulation_Template.py</code>	Base class every <code><Name>Simulation</code> implementation follows; defines the common step fields (<code>timeStamp</code> , <code>temperature</code> , <code>pressure</code>).
<code>simulation<Name>.py</code>	One file per reaction, e.g. <code>simulationNaCl2.py</code> → class <code>NaCl2Simulation</code> .
<code>reaction_definitions/<Name>.yaml</code>	Reaction-specific configuration (default starting amounts, etc.) loaded by the matching simulation class.